shift.AI

PLAN SMARTER. BUILD WHAT MATTERS.

SOFTWARE REQUIREMENTS SPECIFICATION

Implementation-ready specification for the hackathon build

Version 1.0
Project shift.AI
Core stack Next.js + TypeScript | FastAPI + Python | Gemini API | MongoDB Atlas | LangGraph
Brand #FF7A00 | #FFF4E6 | #E2E8F0 | #0B1320

> **Primary build target**
>
> After the developer installs dependencies and places valid GEMINI_API_KEY and MONGODB_URI values in backend/.env, the core application must run without code edits. Hosting/deployment is intentionally excluded from this SRS.

Document Control

Field	Value
Document	shift.AI Software Requirements Specification
Version	1.0
Status	Hackathon implementation baseline
Product goal	Diagnose the business problem first, then decide whether AI is actually needed.
LLM	Google Gemini API
Database	MongoDB Atlas
Backend	FastAPI / Python
Frontend	Next.js / TypeScript / Tailwind CSS / shadcn/ui
Orchestration	LangGraph
Deployment	Out of scope for this document

Contents

1. Purpose and Product Vision

shift.AI is an AI solution discovery, diagnosis, architecture, and decision-support system. Its purpose is to prevent organizations from jumping directly from a vague requirement to an unnecessary AI implementation. The system first gathers context, understands the current business process, reconstructs the workflow, identifies the root cause, and only then decides whether AI is required.

The product is not positioned as a normal chatbot. It is a structured AI strategy workflow that can recommend AI, non-AI automation, process improvement, existing software, API integration, or a hybrid approach.

> **Core product rule**
>
> Never recommend an AI solution simply because the user asked for AI. First establish the real problem, current process, evidence, constraints, and expected business outcome.

1.1 Primary Objective

- Convert vague business requests into clear, evidence-based solution blueprints.
- Ask contextual questions only when information is missing.
- Use uploaded business documents as evidence and avoid asking the user to repeat known information.
- Decide whether AI is required, optional, or unnecessary.
- Challenge the proposed solution using an internal Red Team review.
- Produce a professional final blueprint covering solution design, risks, business value, feasibility, and implementation steps.

1.2 Success Condition

A successful hackathon build allows a user to create a project, describe a problem, answer adaptive discovery questions, upload documents, receive a root-cause and AI-necessity analysis, generate a solution, run an internal Red Team review, and view a final structured blueprint. The project must be runnable locally after configuration values are added to environment files.

2. Product Principles and Differentiators

Principle	Required behavior
Problem before solution	The system must diagnose the actual business problem before architecture generation.
AI is not mandatory	The system may explicitly recommend that AI is not needed.
Adaptive discovery	Questions depend on prior answers, documents, and missing information.
Evidence aware	Recommendations should reference collected facts rather than invented assumptions.
Current-state analysis	The existing people, process, technology, and data environment must be reconstructed.
Red Team loop	A separate review role challenges the proposed solution before finalization.
Business + technical output	The final result combines architecture with value, risk, feasibility, and roadmap.
Project memory	Context is isolated and persistent per project.
Explainability	Important recommendations include a concise why/reasoning statement.

2.1 Standard AI vs shift.AI

Typical assistant:

```
User request -> Immediate answer / AI recommendation

shift.AI:
User request
  -> Discovery
  -> Current-state understanding
  -> Missing information check
  -> Document analysis
  -> Workflow reconstruction
  -> Root-cause analysis
  -> AI necessity decision
  -> Solution design
  -> Red Team challenge
  -> Revision
  -> Business value + feasibility
  -> Final blueprint
```

3. Scope

3.1 In Scope

- Project creation and project dashboard.
- Conversational requirement discovery.
- Adaptive questions and discovery completeness tracking.
- PDF, DOCX, TXT, CSV, and XLSX upload and text extraction.
- Document-aware questioning and retrieval.
- Current workflow reconstruction and bottleneck identification.
- Root-cause analysis.
- AI necessity classification and explanatory score.
- No-AI, automation, software, integration, AI, and hybrid recommendations.
- Solution architecture generation.
- Human-in-the-loop recommendations.
- Red Team review and bounded revision loop.
- Risk, feasibility, and business value analysis.
- Final structured solution blueprint.
- Per-project persistent conversation and analysis state in MongoDB Atlas.
- Professional branded web interface.

3.2 Explicitly Out of Scope for MVP

- Production deployment or hosting configuration.
- Enterprise SSO, OAuth, and complex user/role management.
- Payments or billing.
- Training or fine-tuning a custom foundation model.
- Autonomous execution inside the customer's live business systems.
- Guaranteed financial, legal, medical, or compliance advice.
- Full OCR for image-only scanned documents unless added later.
- Native mobile applications.

4. Users and Primary Use Cases

User type	Need	Expected outcome
Business owner / decision maker	Has a business problem but is unsure what technology is appropriate.	Clear recommendation and business value.
Operations manager	Needs to improve a workflow.	Bottlenecks, root cause, process or automation plan.
Technical lead	Needs a practical architecture.	Components, data, integrations, risks, roadmap.
Hackathon judge / evaluator	Needs to quickly understand the product value.	Visible diagnosis-to-blueprint workflow demonstrating differentiation.

4.1 Core Use Cases

1. Create a new analysis project.
2. Describe a business requirement in natural language.
3. Answer contextual discovery questions.
4. Upload current-process or technical documents.
5. Resume a project without losing prior context.
6. View reconstructed current workflow and bottlenecks.
7. View AI necessity classification and reasoning.
8. View solution architecture and recommended approach.
9. View Red Team findings and risk mitigations.
10. View or copy the final blueprint.

5. End-to-End User Journey

```
Landing page
  -> New Project
  -> Initial problem / requirement
  -> Discovery Agent
  -> Dynamic question loop
  -> Optional document upload
  -> Document Intelligence
  -> Current System + Workflow Analysis
  -> Root Cause
  -> AI Necessity Decision
  -> Solution Architect
  -> Red Team Review
  -> Revision if needed (max 3 cycles)
  -> Business Value + Feasibility
  -> Final Blueprint
```

6. Functional Requirements

6.1 Project Workspace

FR-001 - Create Project

The user shall be able to create a project with a project name and initial problem statement. Industry is optional and may be inferred later.

Priority: Must

FR-002 - Persist Project State

Project stage, discovery progress, messages, documents, analysis, and blueprint data shall be stored in MongoDB Atlas and recoverable after restart.

Priority: Must

FR-003 - Project Isolation

Conversation context and evidence from one project shall not be mixed with another project.

Priority: Must

FR-004 - Dashboard

The dashboard shall show project name, status/stage, last updated date, discovery score, and AI necessity result when available.

Priority: Must

FR-005 - Resume Project

Opening an existing project shall restore prior messages, documents, current stage, and analysis data.

Priority: Must

6.2 Intelligent Requirement Discovery

FR-010 - Do Not Answer Too Early

On a vague requirement, the system shall collect necessary business and process context before proposing a final solution.

Priority: Must

FR-011 - Adaptive Questions

The next question shall be generated from the initial requirement, previous answers, extracted document facts, and missing information.

Priority: Must

FR-012 - Discovery Categories

The system shall track business context, problem definition, workflow, people, technology, data, constraints, impact, integrations, and expected outcome.

Priority: Must

FR-013 - Avoid Repetition

The system shall not ask for facts already available in prior messages or extracted documents unless confirmation is needed.

Priority: Must

FR-014 - One Useful Question at a Time

The default conversational experience should ask one concise, high-value question at a time. Multi-question prompts may be used only when tightly related and more efficient.

Priority: Must

FR-015 - Discovery Completeness

The backend shall maintain structured completeness scores by category and an overall score for UI display.

Priority: Must

FR-016 - Critical Missing Information

A high numeric score shall not force analysis when a critical field required for a reliable recommendation is still missing.

Priority: Must

FR-017 - Question Stop Decision

The discovery controller shall transition to analysis only when it determines that sufficient information exists.

Priority: Must

6.3 Document Upload and Intelligence

FR-020 - Supported Files

The system shall accept PDF, DOCX, TXT, CSV, and XLSX in the MVP.

Priority: Must

FR-021 - File Validation

The backend shall validate file extension/type, file size, and project association before processing.

Priority: Must

FR-022 - Text Extraction

The system shall extract usable text and basic structured content from supported files using Python libraries.

Priority: Must

FR-023 - Chunk and Index

Extracted content shall be cleaned, chunked, tagged with metadata, and stored for retrieval. If vector retrieval is enabled, MongoDB Atlas Vector Search shall be used.

Priority: Must

FR-024 - Document Summary

Each processed document shall receive a concise summary and a processing status.

Priority: Must

FR-025 - Document-Aware Discovery

Relevant facts retrieved from documents shall contribute to missing-information checks and question generation.

Priority: Must

FR-026 - Document Failure Handling

A parsing failure shall mark the document as failed and return a useful user-facing error without crashing the project.

Priority: Must

6.4 Current System and Workflow Analysis

FR-030 - People Analysis

Identify who performs the work, affected roles/departments, and staffing information when known.

Priority: Must

FR-031 - Process Analysis

Reconstruct the current workflow and identify repetitive, manual, slow, duplicate, or failure-prone steps.

Priority: Must

FR-032 - Technology Analysis

Identify current software, tools, databases, integrations, and automation relevant to the problem.

Priority: Must

FR-033 - Data Analysis

Identify available data sources, structure, quality, sensitivity, accessibility, and gaps relevant to the recommendation.

Priority: Must

FR-034 - Workflow Map

Produce a simple ordered workflow representation suitable for display as steps/cards.

Priority: Must

FR-035 - Bottlenecks

Return a structured list of bottlenecks with evidence and impact when available.

Priority: Must

6.5 Root-Cause Analysis

FR-040 - Request vs Root Problem

The system shall distinguish the user's requested solution from the underlying business problem.

Priority: Must

FR-041 - Root Cause Output

Return structured root causes, supporting evidence, confidence, and any unresolved assumptions.

Priority: Must

FR-042 - No Fabricated Facts

Root-cause reasoning must not introduce factual business details that were not provided or reasonably inferred from explicit evidence.

Priority: Must

6.6 AI Necessity Decision

FR-050 - Decision Classes

The system shall classify the problem as AI_REQUIRED, AI_OPTIONAL, AUTOMATION_SUFFICIENT, PROCESS_IMPROVEMENT, EXISTING_SOFTWARE_SUFFICIENT, or HYBRID_SOLUTION.

Priority: Must

FR-051 - AI Necessity Score

Provide an explanatory 0-100 necessity score based on task variability, language/reasoning needs, unstructured data, prediction needs, deterministic alternatives, volume, complexity, and expected value.

Priority: Must

FR-052 - Explain the Score

The UI shall make clear that the score is an advisory heuristic, not a scientific measurement.

Priority: Must

FR-053 - No-AI Recommendation

The system shall be able to explicitly state that AI is unnecessary and provide a practical alternative.

Priority: Must

FR-054 - Alternative Categories

Alternatives may include workflow redesign, traditional automation, API integration, existing SaaS, forms, dashboards, rules engines, database improvement, or other non-AI software.

Priority: Must

6.7 Solution Architecture

FR-060 - Solution Type Decision

The system shall choose among existing software, API integration, custom automation, custom AI, or hybrid solution.

Priority: Must

FR-061 - Architecture Components

For a recommended solution, return required components, component responsibilities, data flow, integrations, human involvement, constraints, and implementation complexity.

Priority: Must

FR-062 - Separate AI and Non-AI Components

The blueprint shall explicitly show which components require AI and which should use deterministic software/automation.

Priority: Must

FR-063 - Human-in-the-Loop

Identify decisions or actions that should require human review, escalation, or approval.

Priority: Must

FR-064 - Assumptions

List assumptions used to design the solution so they can be challenged by the Red Team.

Priority: Must

6.8 Red Team and Revision

FR-070 - Independent Red Team Role

A distinct Red Team agent/prompt shall challenge the proposed solution rather than simply rewriting it.

Priority: Must

FR-071 - Review Areas

Review assumptions, unnecessary AI, privacy, security, hallucination risk, data quality, integrations, edge cases, cost, operational complexity, adoption, and failure modes.

Priority: Must

FR-072 - Structured Findings

Each finding shall include severity, issue, evidence/reason, and mitigation.

Priority: Must

FR-073 - Revision Loop

If material issues exist, the solution architect shall revise the solution using Red Team findings.

Priority: Must

FR-074 - Bounded Iteration

The Red Team revision loop shall stop after at most 3 cycles in the MVP.

Priority: Must

6.9 Business Value and Feasibility

FR-080 - Business Metrics

Estimate potential hours saved, manual steps reduced, workload reduction, response-time improvement, error reduction, scalability, or customer experience improvement when supported by inputs.

Priority: Must

FR-081 - No False Precision

Financial or operational estimates shall be labeled assumptions/estimates when the user did not provide enough exact data.

Priority: Must

FR-082 - Feasibility

Score technical, data, integration, operational, and business feasibility with concise reasoning.

Priority: Must

FR-083 - Risk Assessment

Assess technical, data, security, privacy, operational, adoption, cost, AI reliability, and integration risks using LOW, MEDIUM, HIGH, or CRITICAL.

Priority: Must

6.10 Final Blueprint

FR-090 - Blueprint Generation

Generate a final structured strategy/implementation blueprint after analysis and Red Team review.

Priority: Must

FR-091 - Blueprint Sections

Include problem summary, business context, current system, current workflow, bottlenecks, root cause, AI necessity decision, recommended solution, architecture, AI components, non-AI components, data requirements, integrations, human-in-loop, risks, Red Team findings, business value, feasibility, roadmap, success metrics, and final recommendation.

Priority: Must

FR-092 - Professional Rendering

The blueprint page shall render sections/cards rather than one large chat response.

Priority: Must

FR-093 - Copy and Print

The user shall be able to copy the blueprint and use browser printing. Native PDF export is optional for the MVP.

Priority: Must

FR-094 - Explainability

Important recommendations shall include a short Why/Reason section tied to gathered evidence.

Priority: Must

7. AI Agent and Workflow Requirements

7.1 Agent Responsibilities

Agent	Responsibility	Primary output
Discovery Agent	Understand requirement, identify missing fields, generate next question.	Discovery state + question
Document Agent	Extract and summarize useful facts from uploaded content.	Document facts + summary
Workflow Agent	Reconstruct existing people/process/technology/data flow.	Current workflow + bottlenecks
Root Cause Agent	Separate requested solution from underlying problem.	Root causes + evidence
AI Necessity Agent	Decide whether AI is required and what alternative fits.	Classification + score + reasoning
Solution Architect	Design practical architecture and implementation approach.	Solution proposal

Agent	Responsibility	Primary output
Red Team Agent	Challenge assumptions, risks, gaps, and unnecessary complexity.	Findings + mitigations
Business Value Agent	Evaluate expected value and feasibility.	Value + feasibility
Blueprint Agent	Assemble validated outputs into final structured blueprint.	Final blueprint

7.2 LangGraph State

```
ShiftState
- project_id
- initial_requirement
- messages
- collected_information
- missing_information
- discovery_scores
- uploaded_documents
- retrieved_document_context
- current_workflow
- bottlenecks
- root_causes
- ai_necessity
- proposed_solution
- red_team_feedback
- red_team_cycle
- risk_assessment
- business_value
- feasibility
- blueprint
- current_stage
- errors
```

7.3 Workflow Routing

```
START
  -> load_project
  -> discovery
  -> enough_information?
      NO  -> generate_next_question -> WAIT_FOR_USER
      YES -> document_context
          -> workflow_analysis
          -> root_cause_analysis
          -> ai_necessity
          -> solution_architect
          -> red_team
          -> material_issues?
              YES and cycle < 3 -> revise_solution -> red_team
              NO                -> business_value
```

```
                -> blueprint
                -> END
```

7.4 Required Stage Values

```
NEW
DISCOVERY
DOCUMENT_ANALYSIS
SYSTEM_ANALYSIS
AI_NECESSITY
SOLUTION_GENERATION
RED_TEAM_REVIEW
BUSINESS_VALUE
BLUEPRINT_READY
ERROR
```

8. Document Intelligence and Retrieval

8.1 Processing Pipeline

```
Upload
  -> validate
  -> save temporary file
  -> extract text / rows
  -> clean
  -> summarize
  -> chunk
  -> create metadata
  -> create embeddings (if configured)
  -> store chunks in MongoDB
  -> use Atlas Vector Search for relevant retrieval
```

Vector search should be implemented behind a service abstraction. The application should remain usable for basic document summarization if vector indexing is temporarily unavailable; retrieval-dependent features should return a controlled warning rather than crash.

8.2 Recommended Extraction Libraries

Format	Library
PDF	PyMuPDF
DOCX	python-docx
TXT	Python standard library
CSV	pandas or csv
XLSX	openpyxl or pandas

9. Solution Decision Framework

9.1 Decision Factors

- Task variability and ambiguity.
- Natural-language or reasoning requirement.
- Need to interpret unstructured documents/content.
- Prediction/classification requirement.
- Availability of deterministic rules.
- Existing software capability.
- Data availability and quality.
- Volume and repetition.
- Integration complexity.
- Expected business value compared with implementation cost and risk.

9.2 Decision Output Example

```
classification: HYBRID_SOLUTION
score: 64
summary: AI is useful for natural-language patient queries, but appointment confirmation is deterministic and should use standard automation.
recommended_approach:
- Conversational AI for query understanding
- Rule-based appointment workflow
- Existing scheduling API/database
- Human escalation for sensitive or unusual cases
```

10. Red Team and Risk Review

10.1 Required Review Checklist

- Is AI being used where deterministic software is better?
- Are any architecture assumptions unsupported?
- Could data quality make the solution unreliable?
- Could sensitive data be exposed to the model or logs?
- What happens when the model is unavailable or returns invalid output?
- Are human approvals required for high-impact actions?
- Are integrations realistic and available?
- Are there hidden operational or adoption costs?
- Can the proposal be simplified?
- Are failure modes and edge cases handled?

10.2 Risk Object

```
{
  "category": "AI_RELIABILITY",
  "severity": "HIGH",
  "issue": "Generated answer may be incorrect for uncommon requests.",
  "mitigation": "Use verified business rules, retrieval, and human escalation.",
  "status": "OPEN"
```

```
}
```

11. Business Value and Feasibility

11.1 Business Value Rules

The system may calculate simple estimates from user-provided quantities. It must distinguish known inputs from assumptions. It should not invent revenue, salaries, transaction volume, or time savings when those values were not provided.

```
Known inputs:
- 5 employees
- 2 hours/day spent on repetitive appointment work

Potential modelled result:
- Current workload = 10 staff-hours/day
- Estimated post-solution workload = 2.5 staff-hours/day (assumption)
- Potential reduction = 7.5 staff-hours/day (estimate)
```

11.2 Feasibility Model

Dimension	Meaning
Technical	Can the solution be built with available technology and skills?
Data	Is enough accessible, reliable data available?
Integration	Can the solution connect to required systems?
Operational	Can staff realistically use and maintain the solution?
Business	Does expected value justify effort, risk, and cost?

12. Final Blueprint

12.1 Required Blueprint Sections

11. Executive Problem Summary
12. Business Context
13. User Request
14. Current System
15. Current Workflow
16. Bottlenecks
17. Root Cause
18. Evidence and Assumptions
19. AI Necessity Decision
20. Recommended Approach
21. Solution Architecture
22. AI Components
23. Non-AI Components

24. Required Data
25. Integrations
26. Human-in-the-Loop Design
27. Risk Assessment
28. Red Team Findings
29. Revised/Final Solution
30. Business Value
31. Feasibility
32. Implementation Roadmap
33. Success Metrics
34. Final Recommendation

12.2 Blueprint Quality Rules

- Use structured sections and concise language.
- Clearly label assumptions and estimates.
- Explain why each major recommendation exists.
- Do not hide Red Team findings; show material risks and mitigations.
- If AI is unnecessary, the blueprint must still be complete and implementation-oriented.
- Architecture diagrams may be represented as clean UI flow blocks in the MVP.

13. Frontend Requirements

13.1 Technology

```
Next.js
TypeScript
Tailwind CSS
shadcn/ui
```

13.2 Required Routes

```
/                    Landing page
/dashboard           Project dashboard
/project/new         Create project
/project/[id]        Main workspace / overview
/project/[id]/documents   Documents
/project/[id]/analysis    Analysis
/project/[id]/solution    Proposed solution + Red Team
/project/[id]/blueprint   Final blueprint
```

13.3 Main Workspace Layout

```
+-------------------------------------------------+
| shift.AI                    Project Name  |
+-----------+-------------------------------------+
| Overview  |                                     |
| Discovery |       Conversation / Stage          |
| Documents |                                     |
```

```
| Analysis   |                          |
| Solution   |                          |
| Red Team   |                            |
| Blueprint  |                          |
+------------+------------------------------------+
| Ask shift.AI...                Upload     Send |
+-------------------------------------------------+
```

13.4 Required UI States

- Loading project.
- Sending message.
- Processing document.
- Analyzing current workflow.
- Evaluating AI necessity.
- Generating solution.
- Red Team reviewing solution.
- Building blueprint.
- Configuration missing.
- Recoverable error.
- Empty/no-project state.

13.5 Discovery Progress UI

The workspace shall show per-category progress such as Business, Workflow, Technology, Data, Constraints, and Overall. The UI must not imply mathematical certainty; use the score as a completeness indicator.

14. Design System

14.1 Brand Colors

Token	Hex	Use
Primary Orange	#FF7A00	Primary CTA, active state, small accents, progress.
Warm Secondary	#FFF4E6	Highlighted cards and warm background sections.
Accent Blue-Gray	#E2E8F0	Borders, dividers, neutral cards, secondary surfaces.
Dark Navy	#0B1320	Primary text, navigation, strong contrast backgrounds.

14.2 Visual Direction

- Professional, minimal, premium, clean, strategic, and human-designed.
- Use generous whitespace and clear hierarchy.
- Use orange selectively rather than flooding the interface.
- Prefer simple cards, borders, and typography over decorative AI imagery.
- Use a clean sans-serif such as Inter in the frontend.

14.3 Avoid

- Neon-purple AI styling.
- Excessive gradients or glow effects.
- Generic AI brain/robot artwork.

- Unnecessary animations.
- Excessive pill-shaped controls and rounded cards.
- Large unstructured walls of generated text.

15. Backend Requirements

15.1 Technology

```
Python 3.11+
FastAPI
Pydantic
PyMongo
LangGraph
Google Gemini API SDK
python-dotenv / pydantic-settings
```

15.2 Backend Principles

- All LLM calls go through one Gemini service abstraction.
- All database access goes through repository/service functions rather than being scattered through route handlers.
- Route handlers validate request/response models and delegate business logic.
- AI outputs used by the UI should be validated as structured Pydantic models where practical.
- Long-running stages should expose clear status values.
- Backend errors must be logged server-side but returned as safe user-facing messages.

16. Project Structure

```
shift.AI/
|
|-- frontend/
|   |-- app/
|   |   |-- page.tsx
|   |   |-- dashboard/
|   |   |-- project/
|   |       |-- new/
|   |       |-- [id]/
|   |           |-- page.tsx
|   |           |-- documents/
|   |           |-- analysis/
|   |           |-- solution/
|   |           |-- blueprint/
|   |-- components/
|   |   |-- ui/
|   |   |-- layout/
|   |   |-- chat/
|   |   |-- dashboard/
|   |   |-- discovery/
|   |   |-- documents/
|   |   |-- analysis/
```

```
|   |   |-- solution/
|   |   |-- blueprint/
|   |-- lib/
|   |   |-- api.ts
|   |   |-- utils.ts
|   |-- hooks/
|   |-- types/
|   |-- public/
|   |-- .env.local.example
|   |-- package.json
|   `-- README.md
|
|-- backend/
|   |-- app/
|   |   |-- main.py
|   |   |-- api/
|   |   |   |-- health.py
|   |   |   |-- projects.py
|   |   |   |-- chat.py
|   |   |   |-- documents.py
|   |   |   |-- analysis.py
|   |   |   `-- blueprint.py
|   |   |-- agents/
|   |   |   |-- discovery_agent.py
|   |   |   |-- document_agent.py
|   |   |   |-- workflow_agent.py
|   |   |   |-- root_cause_agent.py
|   |   |   |-- ai_necessity_agent.py
|   |   |   |-- architect_agent.py
|   |   |   |-- red_team_agent.py
|   |   |   |-- business_value_agent.py
|   |   |   `-- blueprint_agent.py
|   |   |-- workflows/
|   |   |   |-- state.py
|   |   |   |-- routing.py
|   |   |   `-- shift_graph.py
|   |   |-- services/
|   |   |   |-- gemini_service.py
|   |   |   |-- mongodb_service.py
|   |   |   |-- document_service.py
|   |   |   |-- embedding_service.py
|   |   |   `-- vector_service.py
|   |   |-- repositories/
|   |   |   |-- project_repository.py
|   |   |   |-- message_repository.py
|   |   |   |-- document_repository.py
|   |   |   `-- analysis_repository.py
|   |   |-- models/
|   |   |-- prompts/
|   |   |-- utils/
|   |   `-- core/
```

```
|   |       |-- config.py
|   |       `-- constants.py
|   |-- tests/
|   |-- uploads/
|   |-- .env.example
|   |-- requirements.txt
|   `-- README.md
|
|-- .gitignore
`-- README.md
```

> **Mandatory separation**
>
> Do not place Python/FastAPI logic inside the Next.js source tree. The repository must have separate frontend/ and backend/ folders so both sides can be developed and run independently.

17. Gemini API Integration

17.1 Configuration Rule

All Google Gemini configuration must be environment-driven. The developer should fully implement the Gemini service and agent wiring without requiring the project owner to edit source code. Later, the project owner should only need to paste a valid API key and selected model name into backend/.env.

```
# backend/.env
GEMINI_API_KEY=your_key_here
GEMINI_MODEL=your_model_here
```

17.2 Service Abstraction

```
backend/app/services/gemini_service.py

Responsibilities:
- Initialize the Gemini client once
- Read API key/model from configuration
- Provide generate_structured(...)
- Provide generate_text(...)
- Provide embedding method if Gemini embeddings are used
- Apply timeout/retry policy
- Normalize provider errors
- Never expose the API key to the frontend
```

17.3 Missing Key Behavior

The FastAPI application should still boot when GEMINI_API_KEY is missing. AI-dependent endpoints must return a controlled configuration error explaining that the key must be added to backend/.env. This allows the UI, health checks, and non-AI project screens to load before credentials are inserted.

17.4 Structured Output

For discovery decisions, analysis objects, risk findings, feasibility, and blueprint sections, request JSON-like structured output and validate it with Pydantic. If model output fails validation, perform one controlled repair/retry before returning an error.

18. MongoDB Atlas Data Model

18.1 Database

```
Database name: shift_ai
Collections:
- projects
- messages
- documents
- document_chunks
- analyses
- blueprints
```

18.2 projects

```
{
  "_id": ObjectId,
  "name": "Hospital Appointment Optimization",
  "industry": "Healthcare",
  "initial_problem": "...",
  "status": "DISCOVERY",
  "discovery_scores": {
    "business": 80,
    "workflow": 65,
    "technology": 70,
    "data": 50,
    "constraints": 60,
    "overall": 65
  },
  "ai_necessity": null,
  "created_at": ISODate,
  "updated_at": ISODate
}
```

18.3 messages

```
{
  "_id": ObjectId,
  "project_id": ObjectId,
  "role": "user | assistant | system",
  "content": "...",
  "message_type": "chat | question | status",
```

```
  "created_at": ISODate
}
```

18.4 documents

```
{
  "_id": ObjectId,
  "project_id": ObjectId,
  "filename": "workflow.pdf",
  "file_type": "pdf",
  "status": "processing | processed | failed",
  "summary": "...",
  "error": null,
  "created_at": ISODate
}
```

18.5 document_chunks

```
{
  "_id": ObjectId,
  "project_id": ObjectId,
  "document_id": ObjectId,
  "chunk_index": 0,
  "text": "...",
  "metadata": {"filename": "...", "page": 1},
  "embedding": [ ... ]
}
```

18.6 analyses

```
{
  "project_id": ObjectId,
  "business_context": {},
  "current_system": {},
  "workflow": [],
  "bottlenecks": [],
  "root_causes": [],
  "ai_necessity": {
    "classification": "HYBRID_SOLUTION",
    "score": 65,
    "reasoning": []
  },
  "solution": {},
  "red_team": {},
  "risks": [],
  "business_value": {},
```

```
  "feasibility": {},
  "updated_at": ISODate
}
```

18.7 blueprints

```
{
  "project_id": ObjectId,
  "version": 1,
  "content": {
    "problem_summary": "...",
    "root_problem": "...",
    "recommendation": "...",
    "architecture": {},
    "implementation_roadmap": [],
    "success_metrics": []
  },
  "created_at": ISODate
}
```

18.8 Indexes

- projects.updated_at descending for dashboard sorting.
- messages.project_id + created_at for ordered conversation loading.
- documents.project_id for project document listing.
- document_chunks.project_id + document_id.
- Atlas Vector Search index on document_chunks.embedding when vector retrieval is enabled.

19. API Contract

19.1 Required Endpoints

Method	Path	Purpose
GET	/api/health	Health/configuration status.
POST	/api/projects	Create project.
GET	/api/projects	List projects.
GET	/api/projects/{project_id}	Get project state.
DELETE	/api/projects/{project_id}	Delete project and related data.
POST	/api/projects/{project_id}/chat	Send user message and run discovery/next step.
GET	/api/projects/{project_id}/messages	Load conversation.
POST	/api/projects/{project_id}/documents	Upload/process document.
GET	/api/projects/{project_id}/documents	List project documents.
DELETE	/api/projects/{project_id}/documents/{document_id}	Delete document and chunks.

Method	Path	Purpose
POST	/api/projects/{project_id}/analysis/run	Run analysis when discovery is sufficient.
GET	/api/projects/{project_id}/analysis	Get latest structured analysis.
POST	/api/projects/{project_id}/blueprint/generate	Generate/finalize blueprint.
GET	/api/projects/{project_id}/blueprint	Get latest blueprint.

19.2 Chat Response Shape

```
{
  "message": "Approximately how many appointment calls are handled each day?",
  "stage": "DISCOVERY",
  "discovery_scores": {
    "business": 80,
    "workflow": 60,
    "overall": 68
  },
  "needs_user_input": true,
  "analysis_ready": false,
  "project_id": "..."
}
```

19.3 Health Response

```
{
  "status": "ok",
  "database": "connected",
  "gemini_configured": true,
  "vector_search_configured": true
}
```

20. Configuration and Local Setup

20.1 Backend Environment

```
# backend/.env.example
GEMINI_API_KEY=
GEMINI_MODEL=
MONGODB_URI=
MONGODB_DATABASE=shift_ai
CORS_ORIGINS=http://localhost:3000
MAX_UPLOAD_MB=15
```

20.2 Frontend Environment

```
# frontend/.env.local.example
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000
```

> **Secret handling**
>
> Never put GEMINI_API_KEY or MONGODB_URI in NEXT_PUBLIC_* variables. Gemini and MongoDB calls must originate from the backend.

20.3 Required Developer Setup Experience

The repository README files must make setup deterministic. A new developer should not need to manually wire Gemini clients, modify imports, create missing folders, or change source code after cloning.

35. Copy backend/.env.example to backend/.env.
36. Paste GEMINI_API_KEY and MONGODB_URI. Optionally set GEMINI_MODEL if not already given a safe default.
37. Create/activate a Python virtual environment.
38. Install backend/requirements.txt.
39. Start FastAPI.
40. Copy frontend/.env.local.example to frontend/.env.local.
41. Install frontend packages.
42. Start Next.js.
43. Open the local app and create a test project.

20.4 Suggested Local Commands

```
# Backend (Windows CMD / PowerShell example)
cd backend
python -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
uvicorn app.main:app --reload

# Frontend
cd frontend
npm install
npm run dev
```

20.5 MongoDB Driver

The backend requirements must include PyMongo SRV support. The Atlas-provided installation equivalent is:

```
python -m pip install "pymongo[srv]"
```

This command should normally be represented in requirements.txt so future setup is handled by installing the requirements file rather than manually running individual package commands.

21. Error Handling and Resilience

Failure	Required behavior
Missing Gemini key	App starts; AI endpoint returns configuration message.
Invalid Gemini key	Return safe provider/authentication error; no stack trace in UI.
Gemini quota/rate limit	Return retryable message; preserve project state.
Gemini timeout	Apply timeout and limited retry; show recoverable error.
Malformed model JSON	Attempt one repair/retry; validate again; fail safely if still invalid.
MongoDB unavailable	Health endpoint indicates database failure; write-dependent endpoints fail cleanly.
Invalid project ID	Return 404 with concise message.
Unsupported file	Reject before parsing.
Oversized file	Reject using MAX_UPLOAD_MB.
File parsing failure	Mark failed document; project remains usable.
Vector search unavailable	Warn/fallback where possible; do not break unrelated functionality.

22. Security and Privacy

- API keys and database connection strings must exist only in backend environment variables.
- .env and .env.local must be included in .gitignore.
- The frontend must never receive Gemini or MongoDB secrets.
- Uploaded filenames must be sanitized before local temporary storage.
- Validate file size and supported type.
- Do not execute uploaded document content.
- Logs must not print full secrets or MongoDB connection strings.
- Project IDs must be validated and used to scope all document/message queries.
- Deleting a project should also delete related messages, document metadata/chunks, analyses, and blueprints.
- AI-generated recommendations must be presented as decision support, not as guaranteed professional advice.

23. Non-Functional Requirements

ID	Requirement
NFR-001	Core pages should load quickly on a local development environment and provide visible loading states for AI work.
NFR-002	AI operations must not block the frontend without feedback.
NFR-003	The codebase shall use clear separation of routes, services, agents, repositories, models, and UI components.
NFR-004	All major structured AI outputs shall be schema validated before storage/rendering.
NFR-005	The application shall support modern desktop browsers and remain usable at common laptop widths.
NFR-006	UI colors and typography shall meet reasonable contrast/readability expectations.
NFR-007	Project data shall remain consistent after refresh/restart when MongoDB is available.
NFR-008	No model/provider secret shall be committed to source control.
NFR-009	The system shall provide informative errors instead of raw stack traces.

ID	Requirement
NFR-010	The architecture shall make it possible to swap Gemini model names without changing agent logic.

24. Testing Requirements

24.1 Backend Tests

- Health endpoint with and without Gemini key.
- Project create/list/get/delete.
- Message persistence and project isolation.
- Discovery state update.
- Structured output validation and malformed-output handling.
- Document type/size validation.
- Document parsing for each supported format using small fixtures.
- AI necessity classification schema.
- Red Team cycle limit.
- Blueprint generation from mocked structured analysis.
- MongoDB repository behavior using a test database or mocks.

24.2 Frontend Tests / Manual QA

- Landing -> New Project -> Workspace flow.
- Dashboard displays persisted projects.
- Chat messages render in correct order.
- Discovery progress updates after responses.
- Upload shows processing/success/failure states.
- Analysis cards render structured data correctly.
- AI necessity and No-AI decisions are visually clear.
- Red Team findings and severity are readable.
- Blueprint sections render without overflow.
- Missing-configuration messages are helpful.
- Refreshing the browser preserves the current project.

25. Acceptance Criteria

AC	Acceptance test
AC-01	Repository contains separate frontend/ and backend/ roots matching this SRS.
AC-02	Backend starts with FastAPI and exposes /api/health.
AC-03	Frontend starts and communicates with the configured backend URL.
AC-04	With valid MONGODB_URI, a created project persists and reloads after restart.
AC-05	With valid GEMINI_API_KEY, a vague request triggers discovery rather than an immediate final architecture.
AC-06	The next question changes based on previous answers.
AC-07	Uploaded supported documents are processed and shown with status/summary.
AC-08	Information extracted from documents can reduce repeated discovery questions.
AC-09	The system produces current workflow, bottleneck, and root-cause outputs.
AC-10	The AI necessity result can recommend AI, hybrid, automation, process improvement, or existing software.

AC	Acceptance test
AC-11	A No-AI result contains a practical alternative and reasoning.
AC-12	A proposed solution includes architecture, AI/non-AI components, data, integrations, and human-in-loop guidance.
AC-13	Red Team returns structured findings and can trigger revision.
AC-14	Red Team revision is limited to 3 cycles.
AC-15	Risk, business value, and feasibility sections are available.
AC-16	Final blueprint contains all required sections and is not a single wall of chat text.
AC-17	Invalid/missing Gemini configuration does not crash the whole app.
AC-18	Secrets are not exposed in frontend source or NEXT_PUBLIC variables.
AC-19	Brand colors match the supplied shift.AI palette.
AC-20	No hosting/deployment configuration is required by this specification.

26. Out of Scope

The following are deliberately excluded from the hackathon baseline so implementation time stays focused on the differentiated product workflow:

- Hosting, cloud deployment, DNS, CI/CD, and infrastructure-as-code.
- Enterprise authentication/authorization.
- Payment processing.
- Fine-tuning or training a custom LLM.
- Complex multi-tenant billing and organization administration.
- Automatic write access to external hospital/business systems.
- Guaranteed regulatory/compliance certification.
- Native mobile apps.

27. Future Enhancements

- Authentication and multi-user organizations.
- PDF export with branded report templates.
- Interactive architecture diagrams.
- External software/tool catalog and cost comparison.
- Live connectors to customer systems.
- Automatic ROI scenario simulation.
- Additional LLM providers behind the same model-service abstraction.
- Human expert review workflow.
- Project sharing and comments.
- Advanced OCR and image/diagram understanding.

Appendix A - Environment Files

A.1 backend/.env.example

```
GEMINI_API_KEY=
GEMINI_MODEL=
MONGODB_URI=
```

```
MONGODB_DATABASE=shift_ai
CORS_ORIGINS=http://localhost:3000
MAX_UPLOAD_MB=15
```

A.2 frontend/.env.local.example

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000
```

A.3 .gitignore Minimum

```
# Secrets
backend/.env
frontend/.env.local
.env
.env.*.local

# Python
backend/.venv/
__pycache__/
*.pyc

# Node
frontend/node_modules/
frontend/.next/

# Local uploads/cache
backend/uploads/*
!backend/uploads/.gitkeep
```

Appendix B - Recommended Dependencies

B.1 Backend

```
fastapi
uvicorn[standard]
pydantic
pydantic-settings
python-dotenv
pymongo[srv]
python-multipart
langgraph
# Current official Google Gemini Python SDK package
# Pin an appropriate stable version when implementation begins.
PyMuPDF
python-docx
```

```
openpyxl
pandas
```

The developer should use the current official Google Gemini Python SDK available at implementation time and centralize it in gemini_service.py. The rest of the application must not depend directly on provider-specific SDK calls.

B.2 Frontend

```
next
react
react-dom
typescript
tailwindcss
shadcn/ui dependencies
lucide-react
zod (recommended)
```

Appendix C - Core Structured Outputs

C.1 Discovery State

```
{
  "collected_information": {},
  "missing_information": [],
  "scores": {
    "business": 0,
    "workflow": 0,
    "technology": 0,
    "data": 0,
    "constraints": 0,
    "overall": 0
  },
  "critical_missing": [],
  "enough_information": false,
  "next_question": "..."
}
```

C.2 AI Necessity

```
{
  "classification": "AI_REQUIRED | AI_OPTIONAL | AUTOMATION_SUFFICIENT | PROCESS_IMPROVEMENT |
EXISTING_SOFTWARE_SUFFICIENT | HYBRID_SOLUTION",
  "score": 0,
  "reasoning": [],
  "non_ai_alternative": null,
  "recommended_approach": "...",
```

```
  "confidence": 0.0
}
```

C.3 Red Team Finding

```
{
  "category": "...",
  "severity": "LOW | MEDIUM | HIGH | CRITICAL",
  "issue": "...",
  "reason": "...",
  "mitigation": "...",
  "requires_revision": true
}
```

C.4 Feasibility

```
{
  "technical": {"score": 0, "reason": "..."},
  "data": {"score": 0, "reason": "..."},
  "integration": {"score": 0, "reason": "..."},
  "operational": {"score": 0, "reason": "..."},
  "business": {"score": 0, "reason": "..."},
  "overall": 0
}
```

Appendix D - Definition of Done

> **Hackathon Definition of Done**
>
> The build is considered implementation-ready when the repository can be cloned, dependencies installed, environment templates copied, and the core experience works after valid Gemini and MongoDB credentials are inserted - without source-code edits.

[ ] Frontend and backend are separate and start independently.
[ ] Environment templates are included and real secret files are ignored.
[ ] MongoDB project persistence works.
[ ] Gemini service is fully wired from environment configuration.
[ ] Adaptive discovery works and does not immediately jump to a solution.
[ ] Document processing works for the supported MVP formats.
[ ] Workflow, bottleneck, and root-cause analysis render in the UI.
[ ] AI necessity decision supports No-AI outcomes.
[ ] Solution Architect produces structured architecture.
[ ] Red Team challenge + bounded revision works.
[ ] Risk, business value, and feasibility are included.

- [ ] Final blueprint renders professionally.
- [ ] Key failure states are handled cleanly.
- [ ] UI uses the supplied shift.AI brand palette.
- [ ] README contains complete local setup steps.
- [ ] No hosting/deployment section or requirement is included.